QR PACKING APP



Created by: Group 6

- Ngô Quang Huy - 2101040101
- Triệu Duy Thái - 2101040162
- Vũ Ngọc Cường - 2101040047
- Lê Đức Tài - 2101040160
- Nguyễn Đức Hoàng Sơn- 2101040154

Topic’s content

1. **Project Overview**
   Description, Inspiration, Main Features, Technologies
2. **Planning & Tasks Delivery**
   Specific Individual Tasks
3. **Software’s operations process**
   Details of code implementation process.
   Key outputs of the process.
4. **Demo And Practical application**
   Examples of real-life usage.
5. **Conclusion**
   To sum up the Topic

PROJECT OVERVIEW

PROJECT DESCRIPTION

- QR Packing App - application used in the packaging industry
- Allowing users to systematically record the packing process and save the packing videos in order to ensure quality control
- Helping to provide evidence for shipment disputes, and improve overall transparency and traceability in logistics operations.



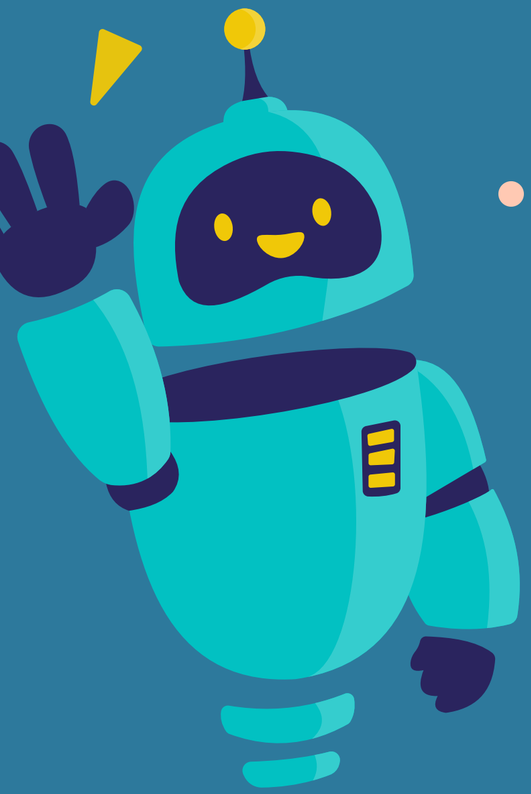
INSPIRATION

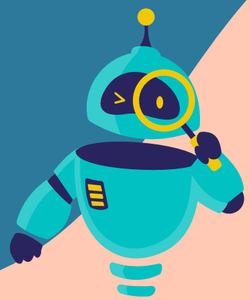
- Observed frequent issues in logistics: missing items, shipment disputes, and lack of transparency.
- Recognized the need for a reliable way to record the packing process.
- Aspired to combine QR code technology with video recording for better traceability.
- Aimed to increase accountability and efficiency in packaging operations.
- Wanted to bridge the gap between manual work and digital documentation.
- Driven by the vision of building smarter and safer supply chains.



MAIN FEATURES

- Scan QR Code to get ProductCode
- Recording video based on received QR Code
- Save recorded videos on history so as to review





USED TECHNOLOGIES



FE:

- ReactNative
- Expo tools (Camera, Video, ...)

BE:

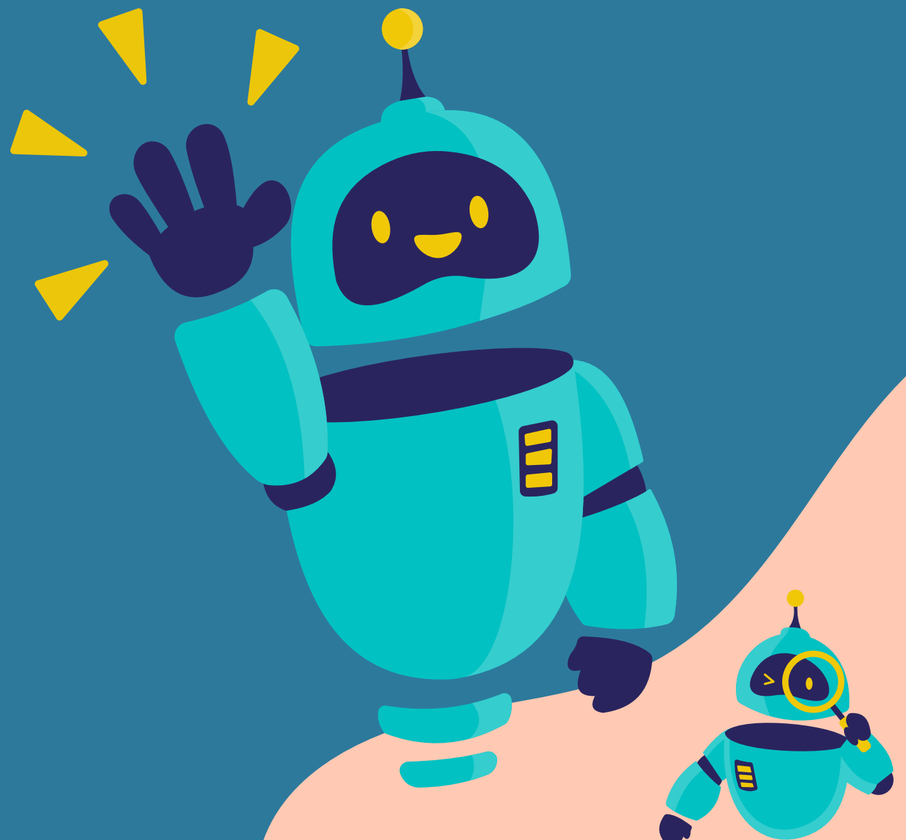
- ASP .NET Core
- EF Core

Database:

- Postgres

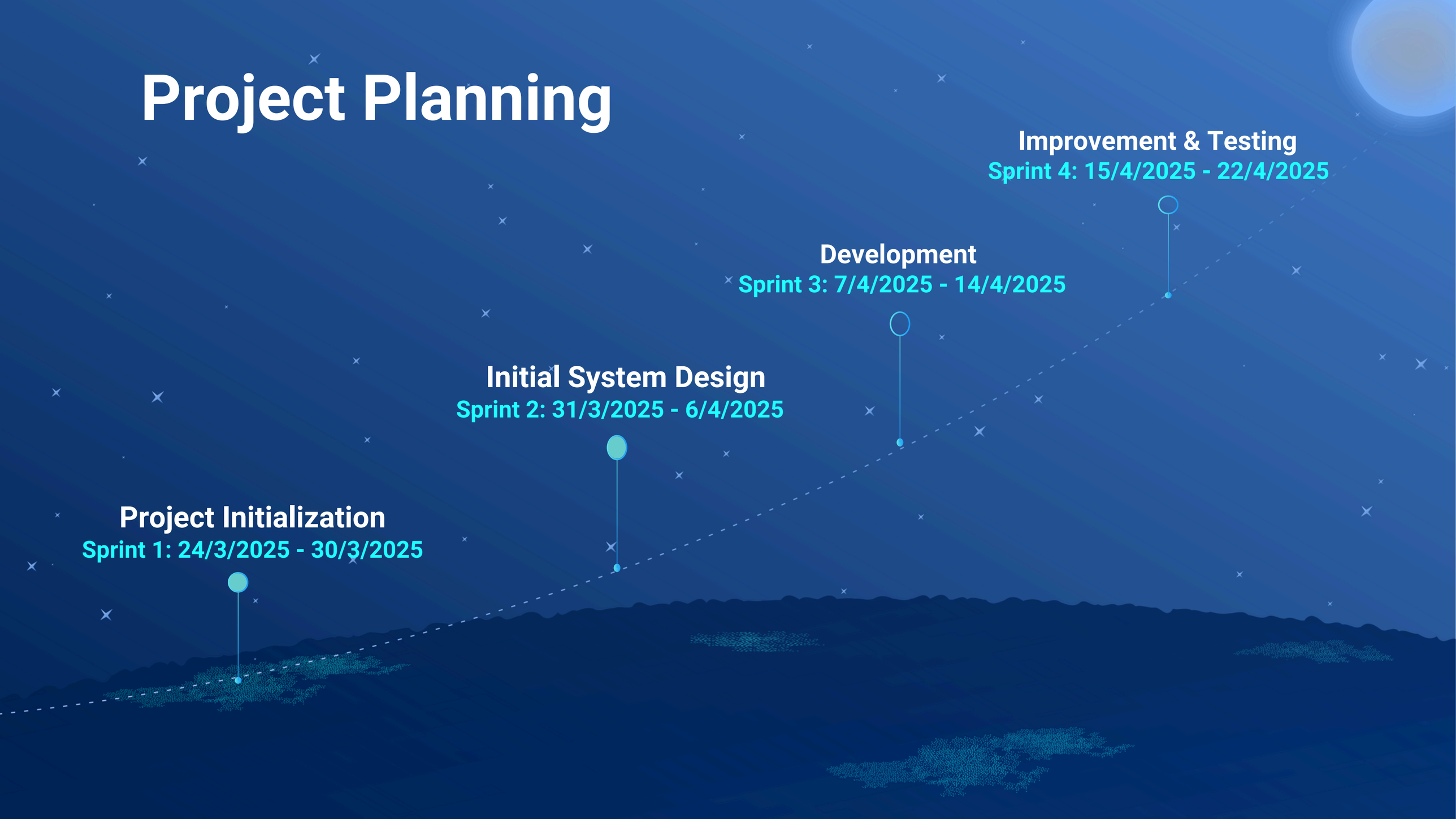
Tools:

- Dbeaver
- VisualStudio, AndroidStudio, ExpoGo



PLANNING & TASKS DELIVERY

Project Planning



Tasks Delivery

	Huy (Leader)	Thái (FE Dev)	Sơn (BE Dev)	Cường (BE dev)	Tài (FE Dev)
Sprint 1 (24/3/2025 - 30/3/2025)	• Planning • Tasks delivery • Set up dev environment	Exploring the approach to project implementation	Exploring the approach to project implementation	Exploring the approach to project implementation	Exploring the approach to project implementation
Sprint 2 (31/3/2025 - 6/4/2025)	• Design Database • Build Skeleton for BE Core	• Build Skeleton for FE Core • Design UX/UI	• Design Database • Ready to implement Authentication Modules in the next sprint	• Design Database • Ready to implement Authentication Modules in the next sprint	• Design UX/UI • Ready knowledge to implement Call API Authentication to view
Sprint 3 (7/4/2025 - 14/4/2025)	Support core modules QR Scanner Modules	QR Scanner Modules: • QR Scanner • Video Recording • Setting Screen	• Authentication Modules implementation • Create API Get/Post videos	• Create API Get/Delete video by id • Create Get ProductByQRCode	Code UI and Call API for: • Authorization • MainScreen • VideoHistory • VideoDetails
Sprint 4 (15/4/2025 - 22/4/2025)	Improvement And Testing	Continue Fixing Bug and improve UI/UX	Continue creating sub-API and help to integrate API to FE	Continue creating sub-API and help to integrate API to FE	Continue Fixing Bug and improve UI/UX

SOFTWARE'S OPERATIONS PROCESS

BACK-END IMPLEMENTATION

Core Back-End

QRPackingApp.Business:

- Contains business logic and rules
- Coordinates between data access and API layers.
- Handles service-level operations.

QRPackingApp.Core:

- Contains API Result and BaseController for responses

QRPackingApp.Data:

- Contains Repositories , add Repositories Service Registration
- Data Seeding
- Migrations

QRPackingApp.DTO:

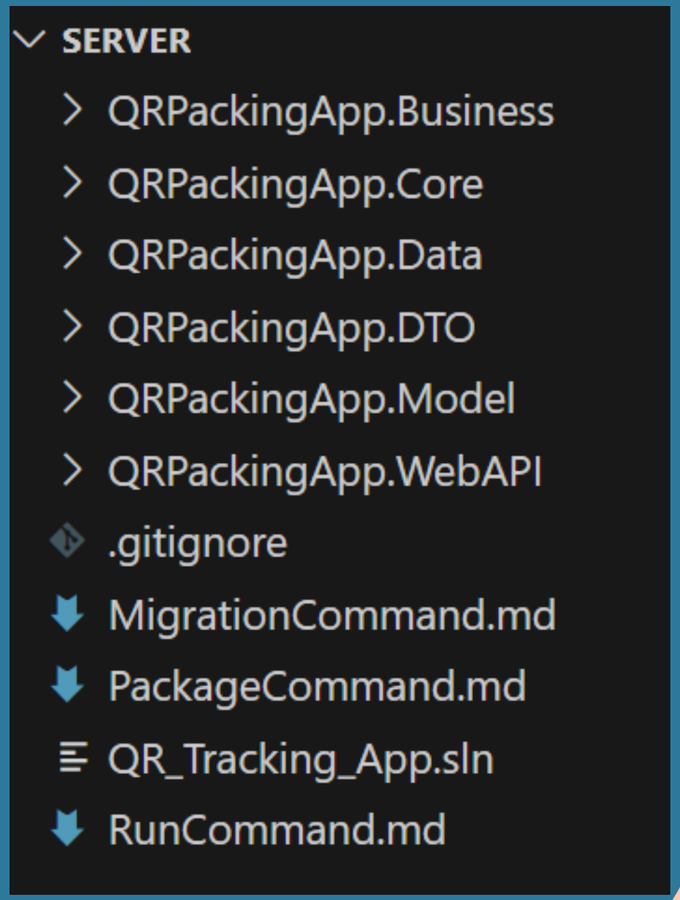
- Holds Data Transfer Objects.
- Used to transfer structured data between layers, especially from Business to WebAPI.
- Helps in shaping responses for the client.

QRPackingApp.Model:

- Defines domain models/entities representing the database structure

QRPackingApp.WebAPI

- Exposes RESTful endpoints to the client or external services.
- Acts as the entry point of the application.
- Handles HTTP requests and responses.



DATABASE, MODEL CREATING



Core Entities:

- users: Manages user authentication and role-based access.
- products: Stores product-specific information linked by unique QR codes.
- videos: Records metadata linking products, users, and associated video files with timestamps.

Relationships:

- Users and products have a one-to-many relationship with videos, enabling easy retrieval of recorded packaging videos.

Authentication

POST /api/Auth/login

- User submits login credentials (username/password)
- Server validates the credentials and generates a JWT token
- Client sends the token in the request header:
- Authorization: Bearer <token>
- Server validates the token and grants access to protected resources

Upload Videos API

POST /api/Video/Upload

- Client sends a video file and metadata via HTTP POST (multipart/form-data)
- Authentication: Verify user via JWT token
- Validation: Check file and metadata validity
- Save video file to server storage (wwwroot/Videos)
- Generate public URL for the uploaded video
- Store video metadata in the database
- Respond to client with success message and video URL

Get Video By Product Id

GET /api/Video/get-by-product/{productId}

- Client sends a GET request to the API
- User is authenticated via JWT
- Query video list by product ID
- Return video list as response

Get Video Details

GET /api/videos/{id}

- Client requests detailed info of a specific video by its ID
- Authentication: Required
- Server checks if the video exists in the database
- If found, responds with:
- Video file path
- Product and user details
- Timestamps (start, end, uploaded)
- Used for viewing or previewing a specific recording

Delete a Video

DELETE /api/videos/{id}

- Client sends a request to delete a specific video
- Authentication: Ensure user is authorized
- Server:
- Validates if video exists
- Deletes file from local storage (wwwroot/Videos)
- Removes corresponding record from the database
- Responds with success message
- Helps users manage and clean up their video history

Get Videos by User

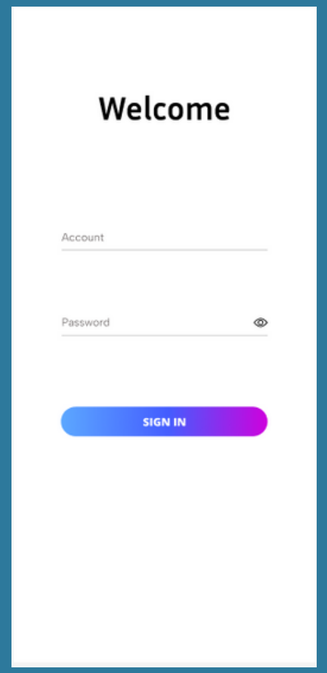
GET /api/videos/get-by-user

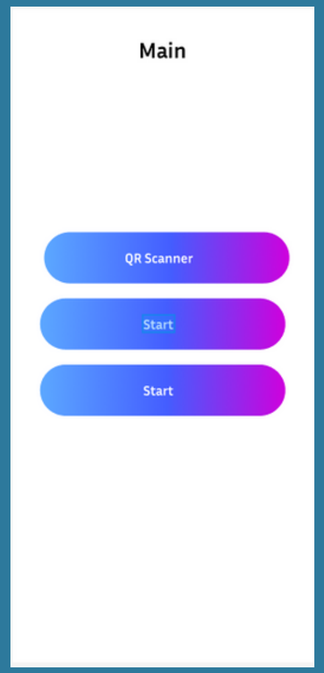
- Client requests video history of the currently logged-in user
- Authentication: User verified via JWT token
- Server identifies user from token context
- Query database for all videos where user_id = current user’s ID
- Returns list with:
- Video ID, product name, timestamps, file path
- Useful for personal video history dashboard

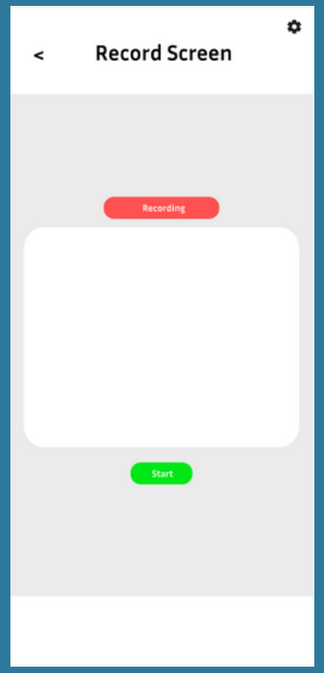
FRONTEND IMPLEMENTATION

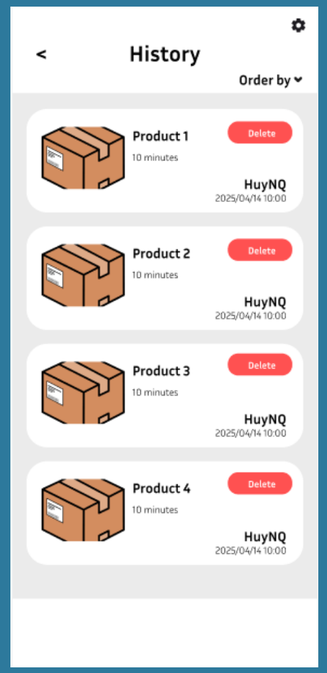
Design phase

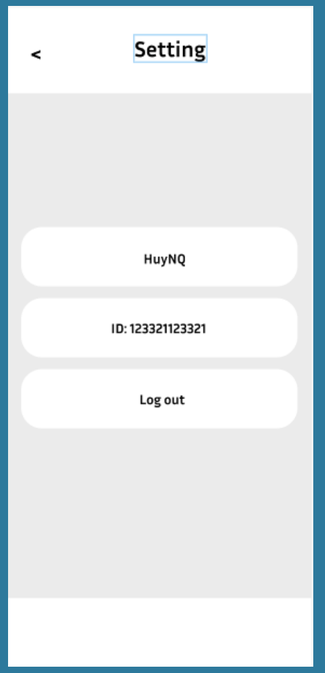
Create a fully functional and simple user-friendly interface











Keeping it **simplest** as possible to match all user's **age** and **familiarity** with technology!

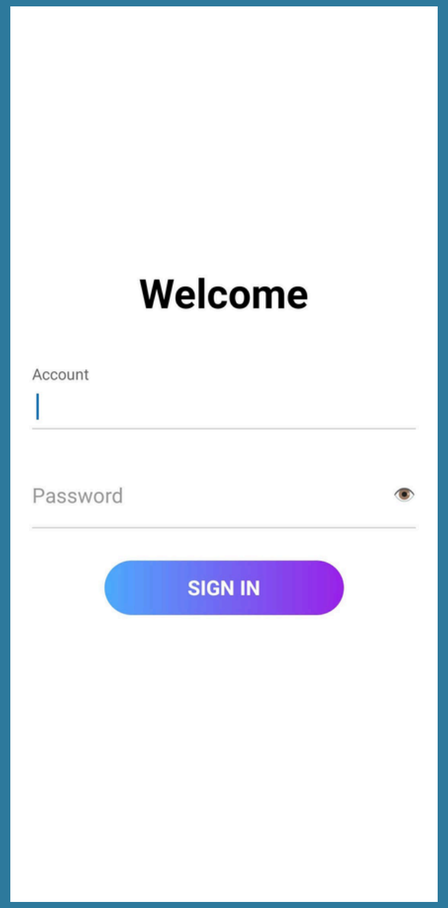
Implement phase

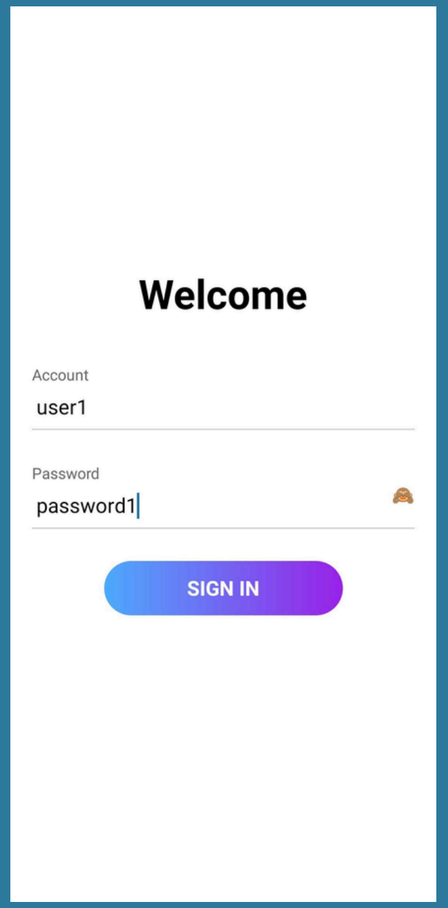
Login Screen

Employee (user) can only logging in with the username and password that company provided.

- Call API from backend to authorize log in.
- Save token to AsyncStorage, handle error and display alert
- Styling and custom component

Login Screen





This is the login screen with a cute monkey that not looking your password :)

Implement phase

Main screen

A simple main screen with buttons to use the application’s features

- Using Stack to create Stack Navigator
- Navigate to other screens

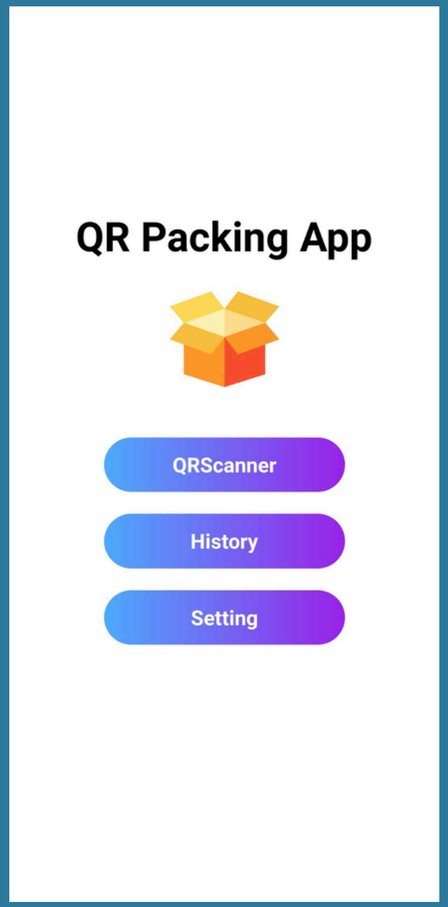
```
<NavigationContainer>
  <Stack.Navigator initialRouteName="Login">
    <Stack.Screen
      name="Login"
      component={LoginScreen}
      options={{ headerShown: false }}
    />
    <Stack.Screen
      name="MainScreen"
      component={MainScreen}
      options={{ headerShown: false }}
    />
```

```
<TouchableOpacity style={styles.button} onPress={() => navigation.navigate('VideoRecording')}
>
  <Text style={styles.buttonText}>QR Scanner</Text>
</TouchableOpacity>

<TouchableOpacity style={styles.button} onPress={() => navigation.navigate('History')}>
  <Text style={styles.buttonText}>History</Text>
</TouchableOpacity>

<TouchableOpacity style={styles.button} onPress={() => navigation.navigate('Setting')}>
  <Text style={styles.buttonText}>Setting</Text>
</TouchableOpacity>
```

Main Screen



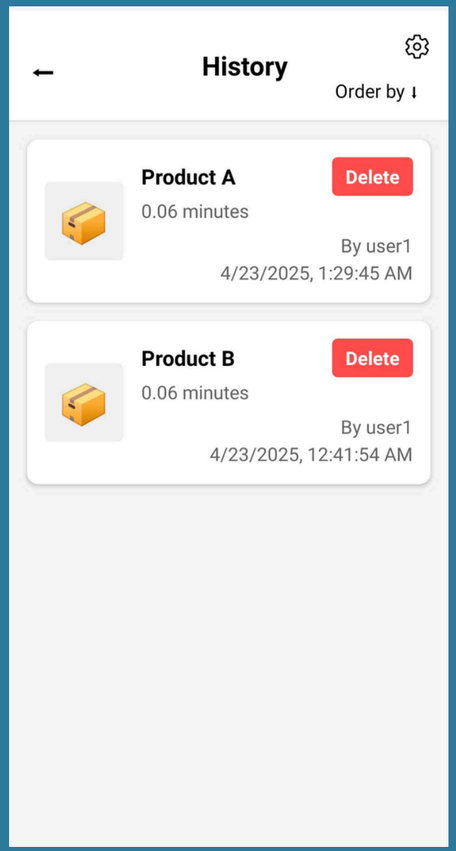
This is our Main screen!

Implement phase

History Screen

Employee (user) can view all the video that they recorded.

- Call API from backend to get videos and delete a video
- Display a list of video with information of that video like name of product, duration, recorded by who and created time.
- Handle logic to sort them desc or asc

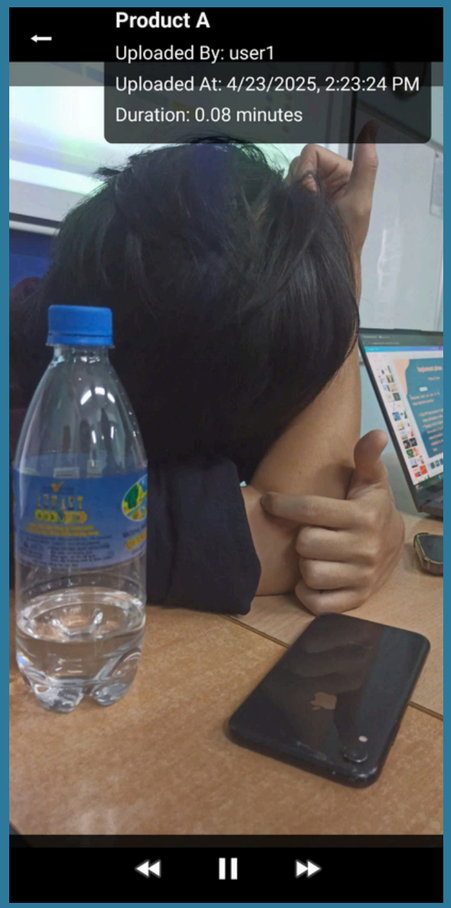


Implement phase

Video Detail Screen

Employee (user) can choose the recorded video that they want to watch.

- Call API from backend to get that video by id.
- Using react-native-video to handle playing the video.
- Display the video with information of that video on the top right of screen



Demo

Product A



Practical application

INCREASE CUSTOMER TRUST

- Record the entire packaging process for transparency
- Provide visual proof to resolve missing or damaged item claims
- Builds long-term trust with customers through openness



SUPPORT COMPLAINT RESOLUTION



- Quickly retrieve videos in case of disputes with customers
- Helps the customer support team handle issues efficiently
- Reduces back-and-forth and improves satisfaction





MONITOR PACKING WORKFLOW

- Managers can remotely review the packaging process
- Ensure adherence to packaging standards
- Detect errors and prevent recurring mistakes



ORGANIZED PACKING VIDEO STORAGE

- Each order is linked to a unique video
- Easy to organize, search, and retrieve past records
- Videos can be integrated into Warehouse Management Systems (WMS)



SEAMLESS WORKFLOW INTEGRATION

- User-friendly interface, easy for staff to adopt
- Compatible with barcode or QR code systems
- Minimal disruption to existing packaging operations

THANK YOU!

Thank you so much for watching our presentation!